PA consistency machines: comparison, source compression, and proof obligations

Comparative construction and verification

13 November 2026

Abstract

We compare the incoming U208 constructions, James Chang's PA209 snapshot, and the published PA196 machine. The nine designated U208 outputs form two rooted transition-graph classes; their common contextual fold is not a set of independent savings. We give a new source representation with 158 distinct internal nodes, compared with PA196's 159, preserving all 3,812 instructions and every ordinal control-flow edge. One representation has maximum depth 37 and maximum zero run six. A second, physically compiled representation uses 196 working states: its six new contextual folds save a reader state, but an additional return-scanner state is needed. This variant passes all 4,959 arbitrary-value execution contracts and exact blank startup. We also give a direct syntactic route to PA196's missing internally stated consistency direction, and counterinterpretations showing that neither membership-rewriting head is derivable from the remaining exact native rules. **No binary Turing machine with fewer than 196 working states has been validated in this investigation.** Source compression does not by itself achieve that target. The packet supplies complete data and finite verifiers so that the established results can be reused without relying on an optimizer's output or confusing a candidate allocation with a correct machine.

1 Scope and result status

The requested endpoint is an explicit deterministic, single-tape, two-symbol machine M, starting on an all-zero tape, with fewer than 196 working states and

$$\mathrm{PA} \vdash \neg\,\mathrm{Halt}(M) \leftrightarrow \mathrm{Con}(\mathrm{PA}).$$

Working states exclude a distinguished halt control. We retain the binary alphabet and blank-input convention of the compared constructions. State labels need not be consecutive, so the largest label is not a state count.

The packet contains both new source representations and a validated new 196-state physical layout. PA196's reader states already implement several pairs of source nodes through contextual sharing. Reparenthesizing the source can destroy those savings or invalidate a tape invariant. A nominal 195-state allocation fails a concrete register-frame test; repairing it requires one additional scanner state. The rejected table is clearly labelled and is not presented as a new record.

There are three different levels of evidence in this packet:

1. Exact finite checks: source/control-flow equality, machine census, transition-graph comparison, startup replay, and complete-tape rejection of tempting merges; also symbolic induction certificates covering all register values and finite spare lists for the new 196-state variant.

2. Mathematical arguments: soundness-preserving source identity, the two independence counter-interpretations, and the direct syntactic consistency proof construction in the accompanying note.

3. Outstanding implementation work: a sub-196 physical allocation, and a newly exported kernel-checked PA theorem for the reverse consistency implication.

The finite experiments neither prove nonhalting by simulation nor establish a lower bound on all possible 195-state machines.

2 Pinned sources and comparison

The PA196 source is pinned to `jamesyc/BusyBeaver@407cdc356f6b586635e71ac92be394255f4bfea4`; the requested PA209 source is pinned to `69d98241a721ead20b4f0b17df9062950b40fbfe`. The incoming archives and the additional comparison/search work are pinned to `VladimirReshetnikov/ProveIt@3a7ddf022dc41b77db860f8db3f888002ddc4f47` [1, 3, 5, 6].

Construction	Working states	Rows	Source sites	Startup steps
PA209	209	418	3,948	58,210
U208 families	208	416	3,909	48,140
PA196	196	392	3,812	58,154
New validated variant	196	392	3,812	59,164

Table 1: The U208 source count is the reported count for the reviewed families. Startup counts for the predecessor and U208 are from the pinned sources; both 196-state startups are independently replayed by this packet.

2.1 What is actually proved in the upstream formal developments

Both James Chang snapshots explicitly export the internal implication

$$\mathrm{PA} \vdash \neg H \longrightarrow \mathrm{Con}(\mathrm{PA}),$$

where H is the exact arithmetic halting sentence for the literal table. Their external nonhalting proofs and standard-model soundness results support the advertised independence conclusions. Those declarations are not themselves a formal export of the biconditional requested here [2, 4].

This is a statement about the theorem inventory, not an assertion that the reverse implication is false. The note `notes/internal_soundness.tex` develops a direct syntactic construction for it. In particular, external validity in a hereditarily finite model is not silently substituted for a uniform PA proof about nonstandard proof codes and nonstandard execution lengths.

2.2 The U208 archives contain two machine families

All eleven incoming ZIP identities were checked against their Git blob identities. Five construction packages designate nine final outputs. Canonically numbering the rooted, action-labelled transition graphs gives two classes, with five and four outputs respectively. There are seven different native byte strings and six native-labelled transition functions. These distinctions account for comments, row order, and state names; they are not nine independent mathematical constructions.

Class A	Class B
ConPA primary	Conditional compatibility
Conditional primary	Entry compatibility
Entry primary	History compatibility
History primary	Research primary
Research plain	

Table 2: The nine designated final U208 outputs. The exact archived paths, hashes, and copied table names are in `data/u208_inventory.json`.

The remaining incoming packages are research dossiers, comparison work, source specializations, and a unary-theory calibration. They were included in the investigation, but were not counted as extra successful transition tables. The historical comparison verifier was also replayed: its 3,744 transition-conjugacy equations, 48,140-step startup calculations, manifest checks, and two negative controls passed. This verifies those finite claims; it is not a new end-to-end PA formalization.

2.3 Why the U208 fold does not simply subtract another state

The U208 constructions perform the same contextual fold of native controls 172 and 211. Equivalent presentations retain different names for the merged control. The proof uses information about recently written tape cells and the incoming-state census. In the Joint/Plain case the merged zero branches take seven and three child steps; the one branches take one. At local boundaries the whole tape is unchanged. Thus the corresponding times satisfy

$$t'(n) = n + 6N_{(172,0)}(n) + 2N_{(211,0)}(n), \qquad n \le t'(n) \le 7n.$$

This is a useful PA-formalizable local equivalence. It applies to that parent table and its verified context invariant. It cannot be subtracted again merely because several archives describe it.

The exact mixed-entry pattern used by that fold is absent from PA196. The newer ProveIt comparison already records this fact and substantial unsuccessful local searches [6]. Consequently a successful combination requires a different contextual certificate or a changed source layout; matching state numbers across the two families has no mathematical meaning.

2.4 What PA196 changes relative to PA209

PA196's reduction is largely a change in the emitted calculus and its representation, not the U208 fold in isolation. Its universal constructor is

$$U(a, B) = 4p(B, a) + 2, \qquad p(u, v) = \binom{u + v + 1}{2} + v,$$

so the pairing arguments are transposed. Implication, membership, and equality retain residues $0, 1, 3$. The matching decoder changes as well. The emitter removes the old seventh equality head, inserts guarded equality vacuity into the separation gate, rearranges heads and antecedents, and uses six contextual pairs in its AX body. Two additional obsolete routing states are removed. The published census is 36 engine states, 13 leaf-routing states and 147 reader states [1].

3 An exact 158-node representation of the PA196 source

3.1 The finite object

The full candidate is `data/source158.json`; its SHA-256 is

`7732de4da7b4c988046c77d9d7d2a6f6dbb7ea5c54a17901190add5b5e5043f0`.

Each row contains a binary path, an instruction opcode, and an operand. The paths are the leaves of a full binary tree in ordinal order. Equal subtrees are stored once when counting the DAG. The five instruction types are increment, conditional decrement, forward break, backward continue, and halt. Nine logical registers are used.

Quantity	Published PA196 source	New representation
Distinct internal nodes	159	158
Instruction occurrences	3,812	3,812
Maximum path length	38	37
Maximum consecutive zero run	6	6
Different resolved instruction rows	0	

Table 3: Exact counts independently checked by `code/verify.py`.

Theorem 3.1 (Exact source identity)**.** *Number the instruction occurrences of both source trees from* 0 *to* 3811*. At every ordinal, the operation, its register operand if any, and every resolved successor ordinal agree. Therefore, for every initial register vector and the same ordinal control, the two source programs have identical register/control executions, including halting.*

Proof. The verifier parses the pinned original Lean tree, independently checks it against `data/source196.json`, and reconstructs the new full binary tree from its paths. It compares the complete ordered instruction words. It then resolves both programs twice. One resolver uses subtree intervals and the relevant ancestor edge; the other uses binary increment, suffix cut, and zero extension to locate the successor leaf. All four 3,812-row graphs agree. No unresolved root exits occur.

For an increment the identical next row and register operation give identical successors. For a decrement, the same register is tested and both the zero and positive successors agree. The two jump forms have identical ordinal targets. Halt occurs at the same ordinal. Induction on the number of source steps proves the claim. This is an identity of source semantics for arbitrary registers, stronger than agreement on a sampled prefix from the clean seed. □

The clean seed remains ordinal 25 and register vector $(2, 0, 0, 0, 0, 0, 0, 0, 0)$. Theorem 3.1 does not transfer the literal tape encoding of that seed to a different physical machine. The separately compiled variant below discharges those obligations for its own source representation and literal table.

3.2 How the representation was found

The search treats the straight-line AX emitter as a word in existing macro blocks and single accumulator increments. It considers repeated substrings as candidate grammar phrases and solves a finite mixed-integer segmentation problem. Activating a phrase and choosing a path through its positions gives a grammar; a phrase split into k children costs $k - 1$ binary grouping nodes. Balanced expansion, followed by semantics-preserving reassociation, supplies a binary source tree.

The one-node gain exploits reuse across the emitter/macro boundary: the pair of adjacent accumulator increments already occurs as `node60` in the multiplication-by-four component of the pairing macro. An AX grammar that reuses that exact subtree can save a full-program DAG node even when its local grammar objective alone does not decrease. Subsequent reassociation restores the six-zero bound and improves maximum depth. The final data and the exact source identity are independent of solver settings or the randomized discovery path.

This search is broader than the earlier bounded one-, two-, and selected three-rotation screens in ProveIt. Its result is an explicit improvement over their 159-node source baseline. It is not a claim that 158 is optimal.

3.3 Why a source-node saving does not automatically save a state

The equation "159 source nodes become 158, therefore 196 states become 195" is invalid. In PA196, several distinct nodes share a reader state because their incoming tape contexts distinguish them. A new tree may retain fewer such compatible pairs or require extra helper states. The ordinary 158-node source does not preserve all six old AX folds.

The physical allocator must satisfy all of the following jointly: dispatch to the intended instruction, correct arbitrary-register tape behavior, preservation of the register/header layout, and a correct blank-tape startup. A successful source-prefix check is insufficient. Some decoder states are revisited during primitive execution, so treating their rows as freely writable can corrupt a register operation. The next section gives a concrete example of exactly this failure mode.

4 A validated 196-state regrouping and its rejected predecessor

A joint source-and-fold search produces a different 158-internal-node representation, supplied as `data/variant196_source.json`. It retains the same 3,812 ordered primitive instructions and ordinal control-flow edges, has depth 36, and has maximum zero run seven. Six compatible contextual folds give a nominal 195-state table. The entry to the shared copy-guard reader is constrained to retain its incoming one bit.

The nominal 195-state table is nevertheless wrong as a source interpreter. At source ordinal 1003, a zero decrement of register 1 begins with seven trailing zero address bits. With gap 48, nine zero registers and spare list [0], the machine first misroots after 81 steps. After 276 steps it reaches a clean root with gap 49 and spare list $[0, 1]$, rather than the specified unchanged file and gap. The seven-zero return scanner has mistaken a path region for the return delimiter. The complete tape counterexample is `data/rejected195_example.json`; it is replayable by `code/check_concrete.py`. This refutes the intended first-return-to-clean-root instruction contract, not an eventual blank-tape halting equivalence by itself. With register 1 initially equal to 1, the same candidate reaches a wrong clean root after 278 steps: it leaves register 1 equal to 1 rather than decrementing it to 0, and takes the zero-branch successor. This also violates the logical register effect, independently of the padding and spare-list discrepancy.

The repair changes native row 34/0 from $(0, L, 75)$ to $(0, L, 2000)$ and adds

$$\delta(2000, 0) = (0, L, 75), \qquad \delta(2000, 1) = (1, L, 2).$$

The new scanner distinguishes eight consecutive zeros. The repaired machine has 196 working states and is supplied in both native and dense forms, `data/variant196.native.tm` and `data/variant196.tm`. In dense form its start is 0, its working states are $0, \ldots, 195$, and its separate halt is 196. The native table's SHA-256 is

`f6594edc95efed02f26b88f71c9b1f7699965a258739836f24e6b641887b0cd9`.

All 4,959 symbolic instruction cases pass with gap $48 + g$, for arbitrary $g \in \mathbb{N}$, nine independent natural register values, an arbitrary first spare value, and an arbitrary finite remaining spare list. The certificates use 1,177,411 symbolic inferences and cover all 392 table rows. Their durations have nonnegative affine coefficients and positive constant terms; the smallest constant is 3. Unary self-loop induction and four rederived one-block traversal lemmas handle unbounded register values and lists. These are executable mathematical certificates, not finite small-register sampling and not proof-assistant kernel certificates.

From the blank tape, the repaired machine reaches exactly the baseline clean frame after 59,164 steps: native root 84, head -2, the same 66 one-cells, gap 48, source ordinal 25, registers $(2, 0, \ldots, 0)$, and 26 spare blocks of value 1. Startup and runtime are checked separately. The standalone note `notes/regrouped196_validation.tex` explains certificate soundness and the PA-formalizable halting transfer in detail.

Theorem 4.1 (Halting transfer for the repaired variant)**.** *Let R be the repaired table and M_{196} the pinned baseline. The exact source identity, startup checks and uniform positive runtime contracts, with the elementary arithmetic soundness proof of their certificate rules, give the paper-level conclusion*

$$\mathrm{PA} \vdash \mathrm{Halt}(R) \leftrightarrow \mathrm{Halt}(M_{196}).$$

Consequently the consistency conclusions for the baseline transfer to R.

Proof. A finite source halt gives a physical halt by concatenating startup and primitive contracts. Conversely, if a literal halt occurred at time T but the source did not halt, $T + 1$ source steps and their positive physical segments would give a literal trace longer than T with a working endpoint. Determinism contradicts the missing outgoing transition at the alleged halt. These arguments are finite-sequence inductions expressible in PA. Apply them to both implementations of the identical ordinal source. □

The new layout is a successful intermediate result at the existing bound. Its extra scanner state consumes the reader-state saving; it does not satisfy the requested strict inequality.

5 A complete-tape rejection of a plausible merge

One near miss in the existing local searches identifies dense states 96 and 103, corresponding to native controls 109 and 124. It is tempting because local representative contexts look compatible. We tested the actual full root-frame primitive contract.

Take source ordinal 16, path `0100110110`, whose instruction is increment logical register 6. Use header gap 56, all nine registers zero, and spare list [0]. Redirect native 109 to 124 and otherwise retain the table. The exact results are:

Quantity	PA196	Merged candidate
Steps to the displayed successor frame	596	574
Successor path	`0100110111`	`0100110111`
Incremented register	6	8
Final spare list	$[0, 1]$	$[0, 1]$

The complete tapes are checked, including every register block. Thus a correct program location does not imply a correct implementation of the source instruction. The script `code/reject_merge.py` reproduces this finite counterexample. It disproves the intended first-return root-frame contract, not the candidate's eventual blank-input halting equivalence.

Another tested shortcut shortened the seven-zero return scan to six by deleting native state 34. Even though six-zero runs in the source paths are trailing, transient tapes also matter. During startup the shortened scan first returns to a different root position on the 54th root visit: the original head is at -2, the altered head at -1, on identical tapes. This is enough to reject the proposed identity simulation; no nonhalting claim is drawn from a finite run.

6 A syntactic route to the internal consistency direction

The accompanying seven-section note `notes/internal_soundness.tex` develops the following route to

$$\mathrm{PA} \vdash \mathrm{Con}(\mathrm{PA}) \to \neg H_{196}.$$

It is a paper-level proof construction, not a newly kernel-checked Lean export. Together with the published internal implication it supplies a mathematical route to the full biconditional for the *196-state* baseline and, by the preceding transfer, the repaired 196-state variant. Neither table is below the requested state bound.

Translate numeric formulas directly into PA, retaining source variable labels:

$$M(a,b) \mapsto \mathrm{Bit}(x_a, x_b), \quad E(a,b) \mapsto x_a = x_b, \quad U(a,B) \mapsto \forall x_a\, \tau(B).$$

The source quantifier's reversed pairing is decoded in the correct order. Keeping labels avoids the binder/property-code mismatch that arises when recoding through the older U208 formula convention. Code zero is truth; the exceptional equality $I(0,0) = 0$ is handled by a fixed logical equivalence. Code one is self-membership, and PA uniformly proves $\neg\,\mathrm{Bit}(z,z)$, so it serves as falsity under every assignment.

The nonlogical core is a uniform proof compiler for finite bit comprehension. For an arbitrary input formula $R(z,\vec{a})$, instantiate full PA induction with

$$Q(n) := \exists s < 2^n\ \forall z < n\ \big(\mathrm{Bit}(s,z) \leftrightarrow R(z,\vec{a})\big).$$

At the successor step, retain s when $R(n,\vec{a})$ is false and choose $s + 2^n$ when it is true. This uses excluded middle, not a decision procedure for R. The proof is generated by fixed syntactic templates, one induction instance, freshening and substitution. It therefore does not define an arithmetic truth predicate.

For the guarded separation gate, use bound $x_a + x_b + 1$ and the selected property

$$(\mathrm{Bit}(x_a, z) \vee z = x_b) \wedge \tau(c)[z/x_b].$$

The case $a = b$ is treated separately because quantification then binds both occurrences in the membership antecedent. A parallel stack retains PA proofs of all live formula codes; a successful Shift halt test produces a PA refutation.

The cleanest endpoint argument works inside PA under the explicit hypothesis Con(PA). That proof-witness invariant keeps every command live. The existing all-model positive scheduler and physical simulation contracts then build literal traces of arbitrarily large length with working endpoints. Determinism excludes a finite halt. The note identifies the scheduler contracts used and avoids confusing an absorbing coded source halt with the literal target's missing outgoing transition.

7 Why neither membership head can simply be deleted

The two heads, up to exchanging antecedents, are

$$L(a,b,d):\ E(a,b)\to(M(a,d)\to M(b,d)),$$
$$R(a,b,d):\ E(a,b)\to(M(d,a)\to M(d,b)).$$

Deleting one and deriving it from pointed separation would shorten the emitter. The note `notes/membership_independence.tex` proves that this particular justification cannot work for the unchanged native rules.

Let σ swap 2 and 3 and fix all other naturals. Put $f_1 = g_1 = \sigma$ and $f_i = g_i = \mathrm{id}$ otherwise. Interpret equality as actual equality. To refute L, interpret

$$M(a,b) := \mathrm{Bit}(f_a(x_a), x_b).$$

Every remaining head, rule and the complete guarded gate is valid, but $M(1,2)\to(E(1,0)\to M(0,2))$ is false when $x_1 = x_0 = 2$ and $x_2 = 0$. To refute R, instead interpret

$$M(a,b) := \mathrm{Bit}(x_a, g_b(x_b)).$$

Then $E(0,1)\to(M(2,0)\to M(2,1))$ is false when $x_0 = x_1 = 2$ and $x_2 = 4$, while all remaining rules and the full gate are valid. The label dependence is legitimate: the raw native calculus does not postulate arbitrary variable-substitution rules for its atoms.

The gate verification is the substantive part. In the first interpretation, form the finite selected bitset and apply f_c^{-1} to obtain the witness at its fresh binder. In the second, use the images of the selected indices under the bijection g_b. The identities $\neg\,\mathrm{Bit}(\sigma(t),t)$ and $\neg\,\mathrm{Bit}(t,\sigma(t))$ handle the diagonal case $a = b$. These constructions work for arbitrary decoded properties, not only a bounded collection of examples.

The finite checker additionally verifies the displayed counterexamples, 27,648 surviving-head instances and 225,000 selected-set constructions. Those samples supplement the universal mathematical proof. The result does *not* exclude a different PA interpretation of a weaker one-head calculus and does not imply a Turing-machine state lower bound.

8 Relation to ConciseIPRA and the remaining optimization problem

The ConciseIPRA material concerns presentations of intuitionistic primitive recursive arithmetic [7]. Its useful design lesson here is to make the syntactic kernel explicit and transport proofs through concrete local rules. Its consistency strength cannot simply replace full PA in this problem. Similarly, the byte size of a Python proof checker does not measure the number of states in its binary, blank-tape implementation.

For the present source, the useful objective is the size of a *complete physical allocation*: source nodes, retained contextual identifications, new helpers, routing, and engine interactions. Minimizing just source length or just DAG nodes can make that objective worse. The 158-node representations and the fully checked 196-state variant are explicit starting points for that combined problem. They have not yet crossed the physical 196-state bound.

8.1 Head-order and antecedent variants

A separate exact symbolic check covers all $6! = 720$ orders of the six nonpropositional AX heads. Independently exchanging the first two antecedents of heads 3, 4 and 6 gives $720 \cdot 8 = 5,760$ variants. The fixed generalization and propositional heads, saved top, guarded gate, parameter reads and cache clearing remain unchanged. The note `notes/axiom_head_permutation.tex` proves that the existing total stack macros remain valid and that replacing each head-selector position by its inverse-permutation position gives a primitive recursive all-model refutation compiler. Antecedent exchange uses the existing propositional head and modus ponens. Thus the logical change is justified before any physical savings are claimed.

A greedy grammar screen over all 5,760 words, followed by eight bounded mixed-integer searches on selected words, did not yield a validated sub-196 table. The best of those eight source grammars has 158 internal nodes but does not immediately supply a geometry-safe physical allocation with the required contextual folds. The solver gaps remain nonzero. These are constructive upper bounds and restricted unsuccessful searches, not lower bounds or optimality claims.

9 Reproduction and packet contents

Run from the extracted packet root:

```
python3 code/verify.py
python3 code/reject_merge.py
python3 code/reject195.py
python3 code/check_variant.py
python3 code/check_membership_countermodels.py
```

The principal verifier needs Python's standard library only. It reparses the original source, checks the two source graphs independently, verifies the table counts and U208 classes, and replays all 58,154 startup steps against the exact complete tape. Its output is reproducible and is recorded in `data/verification.json`. For the new 196-state table, separately replay:

```
python3 code/check_startup.py -machine data/variant196.native.tm
  -output certificates/variant196/startup.json

python3 code/check_runtime.py -machine data/variant196.native.tm
  -source data/variant196_source.json -gap 48 -states 196
  -output certificates/variant196
```

Each displayed wrapped command is one shell invocation. Do not disable Python assertions with `-O` or `PYTHONOPTIMIZE`. The checkers return a nonzero exit code on failure. The runtime replay normally takes about one to two minutes in the environment used for this investigation.

The packet contains the original PA196 and PA209 tables, all nine designated U208 native tables, exact source data, provenance and licenses, the independent verifiers, and five LaTeX/PDF documents. The supporting notes contain the longer logical and runtime proofs. The included 195-state table is explicitly rejected by a concrete contract failure; only the repaired 196-state variant is validated. The alphabet, blank-input convention, state-count convention and requested logical strength have not been weakened.

References

[1] J. Chang. *Nonhalting and PA-independence for a 196-state Turing machine*. Pinned revision 407cdc356f6b586635e71ac92be394255f4bfea4.

[2] J. Chang. *PA196 formal theorem statements*. Pinned theorem inventory.

[3] J. Chang. *Nonhalting and PA-independence for a 209-state Turing machine*. Requested PA209 snapshot.

[4] J. Chang. *PA209 formal theorem statements*. Pinned theorem inventory.

[5] V. Reshetnikov, ProveIt repository. *Incoming consistency-machine archives*. Pinned archive collection.

[6] ProveIt. *PA196 and the incoming U208 construction*, with associated source, allocation and local-fold experiments. Pinned comparison subtree.

[7] ProveIt. *ConciseIPRA*. Pinned proposals and formal developments.